On the Effect of Instrumentation on Test Flakiness

Shawn Rasheed*, Jens Dietrich†, Amjed Tahir‡
*Universal College of Learning, Palmerston North, New Zealand
†Victoria University of Wellington, Wellington, New Zealand
‡Massey University, Palmerston North, New Zealand
jens.dietrich@vuw.ac.nz; s.rasheed@ucol.ac.nz; a.tahir@massey.ac.nz

arXiv:2303.09755v1 [cs.SE] 17 Mar 2023

***Abstract*—Test flakiness is a problem that affects testing and processes that rely on it. Several factors cause or influence the flakiness of test outcomes. Test execution order, randomness and concurrency are some of the more common and well-studied causes. Some studies mention code instrumentation as a factor that causes or affects test flakiness. However, evidence for this issue is scarce. In this study, we attempt to systematically collect evidence for the effects of instrumentation on test flakiness. We experiment with common types of instrumentation for Java programs—namely, application performance monitoring, coverage and profiling instrumentation. We then study the effects of instrumentation on a set of nine programs obtained from an existing dataset used to study test flakiness, consisting of popular GitHub projects written in Java. We observe cases where real-world instrumentation causes flakiness in a program. However, this effect is rare. We also discuss a related issue—how instrumentation may interfere with flakiness detection and prevention.**

***Index Terms*—Flaky Tests, Test Bugs, Instrumentation**

I. Introduction

A test can only provide useful feedback if it consistently has the same outcome (either pass or fail) for every execution with the same code version. Flaky tests may pass in some runs and fail on others. Test flakiness has been gaining the attention of both academia and industry because of its negative impact on testing, testing-dependent processes (especially automated testing in CI/CD pipelines), and techniques that rely on executing tests [1], [2]. Several factors can cause test flakiness, including concurrency, test order dependency, network, shared state and platform dependencies. Most of these causes are common across programming languages and platforms [3]–[5].

One potential factor that may have an impact on test flakiness is instrumentation. Ideally, for common use cases of instrumentation, such as code coverage, the effect of instrumentation should be transparent to the application. Concerning test flakiness, this means that test outcomes should remain the same with or without instrumentation. This may well not be the case in practice, though. A study on code coverage at Google [6] describes flakiness as a cause for failed coverage computation. Lam et al. [7] explain how their instrumentation for root-causing flakiness interferes with program behaviour and leads to increased/decreased test flakiness.

However, there are gaps in these studies with respect to the question we are interested in. Those studies are not focused on the effect of instrumentation on flakiness, and the full datasets are not available as the studies are from the industry. This work is an attempt towards addressing this, in which we discuss the effect of instrumentation on test flakiness, and how it affects flaky test prevention/detection techniques. We propose evaluation metrics and perform a preliminary evaluation on a dataset used in a previous test flakiness study to determine whether instrumentation impacts flakiness. In this work, we address the research question:

RQ: Does instrumentation increase/decrease test flakiness?

II. Related work

A. Flaky test detection

Several techniques have been proposed to detect flaky tests, most of which determine flakiness by observing transitions of test outcome across multiple runs [8], [9]. There are two main approaches used for detecting flaky tests: static techniques that rely only on analyzing the test code without actually executing tests or dynamic techniques that involve the execution of tests [2]. Most of the existing tools focus on specific causes of test flakiness, such as concurrency (e.g., Shaker [10]) and test order-dependency (e.g., iFixFlakies [11]). There have been recent attempts to build lightweight static approaches for flaky test prediction that aim to avoid or minimize test reruns [12], [13].

B. Instrumentation and flakiness

Wing et al. [7] used instrumentation to record runtime properties for root-causing flaky tests. They reported that their instrumentation could change runtime behaviour and thus decrease or increase test flakiness. In their study, runtime overhead from instrumentation was observed to affect the reproducibility of flaky tests (by executing a random sample from 59 flaky tests, two tests were flaky only with instrumentation and three were flaky without instrumentation). Ivanković et al. [6] reported that flakiness due to coverage instrumentation is a common reason for failed coverage computation, which is manifested by performance failure or increased flakiness of non-deterministic tests. Tengeri et al. [14] reported cases where coverage instrumentation changes the behaviour of tests and their results. Finally, Dietrich et al. [15] used instrumentation to intercept network errors to control flakiness caused by dependency on network connectivity.

III. Background

Instrumentation, the process of transparently adding functionality to a program, is often used in dynamic program

analyses. For instance, to capture the runtime properties of a program. Instrumentation in Java often uses bytecode manipulation, facilitated by the availability of bytecode engineering libraries like *asm* and *javassist*. In addition, the Java Virtual Machine (JVM) supports instrumentation directly through agents, which can be deployed statically (via a JVM argument) or attached dynamically.

Instrumentation can cause interference in the behaviour of a program. By interference, we mean that execution is affected by the instrumentation of the program's code. Unavoidably, instrumentation does interfere with available resources as additional instructions use CPU/memory. Some cases of interference include: race conditions caused by timing issues introduced by instrumentation [6]; instrumentation interfering with shared resources; classpath issues where the agent uses a different version of classes that are also part of the classpath of the program and its tests[1].

Interference caused by instrumentation can change the outcome of tests, resulting in test flakiness. We use the following definition of test flakiness here: *tests having different results on multiple executions for the same version of program code; or transitions of test outcomes for the same test across runs*. Here is an example that illustrates flakiness introduced by telemetry instrumentation from MockServer[2]. MockServer features functionality to mock HTTP/HTTPS, REST or RPC services for testing purposes. The test runs MockServer as a JUnit 5 extension, configured with the annotation `@MockServerSettings` to use port 8888 for the mocked service. The same port is used by the OpenTelemetry[3] APM collector service. When the test is executed with OpenTelemetry instrumentation, it fails with the exception that the port is already in use. A dependency on local network resources causes this flakiness.

```
@ExtendWith({
    MockServerExtension.class,
    TestLoggerExtension.class,
})
@MockServerSettings(ports = {8787, 8888})
class MockServerExtensionConstructorInjectionWithSettings ...
```

IV. Study design

This section lists the set of programs used in our study, the types of instrumentation used in the experiments, and the evaluation metrics we use. Our experiment requires rerunning each program's tests 20 times with the baseline (without instrumentation) and then with the use of each of the five instrumentations listed in Table II (total of six).

A. Dataset

We have used the set of programs used in Cordeiro et. al's study on manifesting flakiness by adding noise to the environment [16]. In addition, they are 11 GitHub projects written in Java and use Maven for build automation. For two of the programs (*ozone* and *hbase*), this was not possible due to their size, as they take considerably longer than the other programs (>30min per run). Thus, we have included only nine projects in our analysis (Table I).

[1]In practice, those issues are often avoided by using dependency shading when building agents

[2]https://www.mock-server.com/

[3]https://opentelemetry.io/

TABLE I
Programs with GitHub repository name and commit ID/tag

repository	commit ID/tag
OpenHFT/Chronicle-Queue	bec195b
CorfuDB/CorfuDB	b99ecff
Azure/azure-iot-sdk-java	a9226a5
soabase/exhibitor	d345d2d
vaadin/flow	6.0.6
intuit/karate	09bc49e
mock-server/mockserver	b1093ef
RipMeApp/ripme	19ea20d
killbill/killbill	killbill-0.22.21

TABLE II
Types of instrumentation

inst.	type	version	rep
elastic	APM	1.34.1	/elastic/apm
opentelemetry	APM	1.12.0	/open-telemetry/opentelemetry-java
jacoco	coverage	0.8.8	/jacoco/jacoco/
intellij	coverage	1.0.684	/JetBrains/intellij-coverage
jfr	profiling	N/A	https://docs.oracle.com/en/java/java-components

B. Instrumentation

While it is possible to craft an instrumentation that interferes with a particular test execution, we were interested in studying real-world instrumentation scenarios. To achieve this, we have identified several popular agents used for different purposes – coverage capture, monitoring and profiling.

The instrumentation tools used in the experiment are listed in Table II. This includes Elastic APM, an application performance monitoring system, OpenTelemetry, JaCoCo for Java code coverage, IntelliJ's code coverage, and Java Flight Recorder, which collects profiling data for Java applications.

C. Evaluation Metrics

1) Flaky test count: Flaky test count measures the number of tests over N runs that result in different states across runs for a given instrumentation configuration. If the set of outcomes (as in JUnit) are $\{success, failure, error, skip\}$, a test, t, is flaky across runs (or configurations) if for any two runs, r_1 and r_2, there is a transition across test states, i.e. $r_1(t) \neq r_2(t)$ if we consider test runs as a mapping from tests to states.

2) Flakiness score: The flakiness score measures the variability between test runs. For instance, we may already observe flakiness if a test fails only in 1/20 runs. It is still interesting to see whether we can see this changing to a higher (or lower) value with instrumentation being used, such as 5/20.

For each configuration (baseline or one particular instrumentation) for a program in the dataset, we compute this as follows:

- FT is the set of tests that are flaky in all configurations. Note that this may not include tests that are flaky across configurations. Including those tests in the baseline would

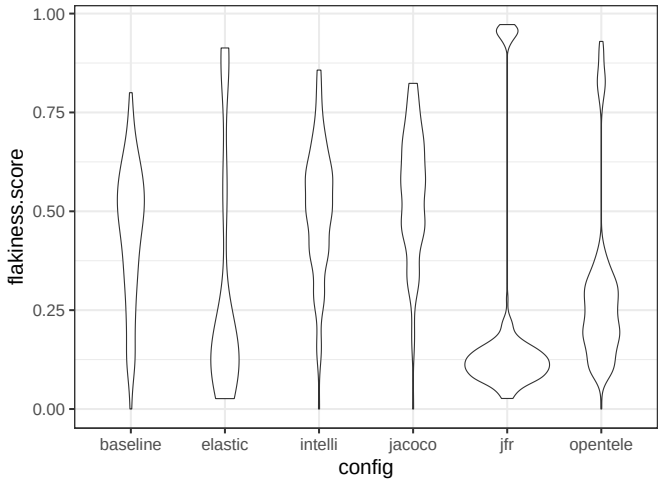


Fig. 1. Variation of flakiness score for *CorfuDB*

be problematic as adding another configuration may affect the flakiness scores for existing configurations.

- For each run i compute a set, r_i consisting of pairs $(t, state) \in FT \times \{pass, fail, error, skip\}$
- For each pair of runs, r_i and r_j, compute the Jaccard distance, $d(r_i, r_j) = 1 - \frac{|r_i \cap r_j|}{|r_i \cup r_j|}$, for N runs. This yields $N(N-1)/2$ values (20 runs resulting in 190 values)
- The distances across runs can then be $aggregated$ (e.g. mean or median) to obtain a flakiness score.

If the flakiness score is 0, there is no flakiness. The flakiness score is greater than 0 if there is flakiness. For example, the baseline for a program's test results has some flakiness (one test fails once), but this one test fails a few more times with instrumentation. In this case, the flakiness score would go up even though the flaky count would remain the same.

D. Experimental setup and process

We run each program's tests 20 times in six different configurations (120 runs for each program), i.e., baseline and the listed five types of instrumentation. Experiments were run on a computer with a 3.2 GHz 6-Core Intel Core i7 CPU with Oracle's Java SE Development Kit 8u301. We made the data from the experiments available online http://www.bitbucket.org/unshorn/inst-study.

V. Results and Discussion

A. Experimental Results

Detailed results for the experiments are shown below. Table III shows the results for the experiments indicating flakiness counts. This includes the count of test outcomes (success, failure/error and flaky) and test runtimes (rt.) for each configuration (i.e., instrumentation). The numbers are averages for 20 runs. Table IV shows the flakiness scores discussed in Section IV-C2 (configurations with zero values are omitted for brevity).

TABLE III
TEST RESULTS FOR ALL CONFIGURATIONS, THE NUMBERS REPORTED ARE THE TESTS RESULTING IN SUCCESS, FAILURE, ERROR OR SKIPPED. TESTS ARE REPORTED ACROSS **20 TEST RUNS**. NUMBERS IN BRACKETS MEANS VARIATION ACROSS RUNS AND REPORT THE MEAN. THE NUMBER OF FLAKY TESTS ACROSS THOSE RUNS ARE REPORTED AS WELL. RUNTIMES ARE IN SECONDS (RT(S)).

program	config	succ.	fail/err	skip	flak.	rt.
ChronicleQueue	baseline	664	1	28	0	(179.2)
	opentele	(663.9)	(1.1)	28	1	(178.8)
	jacoco	664	1	28	0	(251.9)
	intelli	664	1	28	0	(215.2)
	jfr	664	1	28	0	(178.8)
	elastic	663	2	28	9	(190.0)
CorfuDB	baseline	(771.4)	(24.6)	5	15	(2784.2)
	opentele	(768.7)	(27.2)	5	38	(3018.5)
	jacoco	(773.0)	(23.0)	5	10	(2697.5)
	intelli	(773.1)	(22.8)	5	12	(2664.9)
	jfr	(769.2)	(26.8)	5	74	(2632.1)
	elastic	(765.2)	(30.8)	5	75	(2685.6)
azure-iot-sdk-java	baseline	(4302.2)	(0.8)	1	17	(102.3)
	opentele	(4302.8)	(1.5)	1	16	(102.6)
	jacoco	(4303.2)	0	1	0	(102.8)
	intelli	4,305	0	1	0	(102.5)
	jfr	(4304.9)	(0.0)	1	1	(103.1)
	elastic	(4303.2)	(0.7)	1	13	(103.8)
exhibitor	baseline	106	0	0	0	(84.4)
	opentele	106	0	0	0	(88.4)
	jacoco	106	0	0	0	(86.1)
	intelli	106	0	0	0	(85.1)
	jfr	(105.9)	(0.1)	(0.1)	1	(84.4)
	elastic	106	0	0	0	(84.9)
flow	baseline	(4390.5)	(0.5)	9	1	(250.2)
	opentele	(4390.5)	(0.5)	9	1	(265.3)
	jacoco	(4389.5)	(1.5)	9	1	(231.8)
	intelli	(4389.4)	(1.5)	9	2	(239.3)
	jfr	(4390.5)	(0.5)	9	1	(245.9)
	elastic	(4389.5)	(1.5)	9	1	(250.1)
karate	baseline	604	1	0	0	(57.5)
	opentele	604	1	0	0	(57.3)
	jacoco	(603.9)	(1.1)	0	1	(53.3)
	intelli	604	1	0	0	(54.1)
	jfr	604	1	0	0	(53.8)
	elastic	604	1	0	0	(53.9)
killbill	baseline	(1017.9)	(0.5)	(24.6)	65	(133.6)
	opentele	(1017.0)	(0.5)	(25.4)	65	(124.2)
	jacoco	(1029.8)	(0.4)	(12.9)	60	(100.4)
	intelli	(1019.6)	(0.5)	(22.9)	65	(124.4)
	jfr	(1010.2)	(0.5)	(32.4)	65	(140.1)
	elastic	(1017.4)	(0.5)	(25.1)	65	(121.6)
mockserver	baseline	2,870	0	0	0	(92.2)
	opentele	(2867.9)	(1.1)	0	1	(92.7)
	jacoco	(2869.8)	(0.1)	0	3	(92.7)
	intelli	2,870	0	0	0	(91.9)
	jfr	2,870	0	0	0	(92.7)
	elastic	(2869.9)	(0.1)	0	1	(92.8)
ripme	baseline	(87.2)	(72.8)	67	6	(382.6)
	opentele	(86.8)	(73.2)	67	10	(378.0)
	jacoco	(87.7)	(72.3)	67	4	(361.6)
	intelli	(88)	(72)	67	2	(354.4)
	jfr	(87.6)	(72.4)	67	6	(367.5)
	elastic	(87.0)	(73.0)	67	6	(377.7)

In answering our research question if instrumentation changes the test outcome, we observe that, in general, there are no significant changes in test outcomes with or without the use of different instrumentation tools. However, there were cases in some of the programs we examined that showed variation in test outcomes. The two tests that are flaky across configurations, which are in the programs *flow* and *ripme* are due to external flakiness. The test, `FrontendToolsLocatorTest::toolLocated`, in *ripme* fails due to an error in executing in an external program, and `BaraagRipperTest::testRip` fails to access

TABLE IV
FLAKINESS SCORES (MEAN) OVER 20 RUNS FOR EACH PROGRAM/CONFIGURATION (0 VALUES OMITTED FOR BREVITY).

program	config	flakiness-score
ChronicleQueue	opentele	(0.200)
	elastic	(0.339)
CorfuDB	baseline	(0.475)
	opentele	(0.227)
	jacoco	(0.556)
	intelli	(0.484)
	jfr	(0.119)
	elastic	(0.352)
azure-iot-sdk-java	baseline	(0.082)
	opentele	(0.210)
	jfr	(0.200)
	elastic	(0.029)
exhibitor	jfr	(0.200)
flow	baseline	(0.533)
	opentele	(0.467)
	jacoco	(0.467)
	intelli	(0.370)
	jfr	(0.556)
	elastic	(0.533)
killbill	baseline	(0.450)
	opentele	(0.502)
	jacoco	(0.199)
	intelli	(0.435)
	jfr	(0.623)
	elastic	(0.570)
mockserver	jacoco	(0.200)
	elastic	(0.200)
ripme	baseline	(0.518)
	opentele	(0.402)
	jacoco	(0.486)
	intelli	(0.556)
	jfr	(0.474)
	elastic	(0.330)

network resources, which is incidental and not caused by instrumentation. The only confirmed case of flakiness introduced by instrumentation (OpenTelemetry) is the failure of tests in `MockServerExtensionConstructorInjectionWithSettingsMultiplePortTest` in *mockserver* and `FrontendToolsLocatorTest::toolLocated` in *flow*.

On the related question if instrumentation increases test flakiness over multiple runs; again, instrumentation does not appear to have a discernible effect on the stability of flaky tests. However, this is difficult to measure for most programs in the dataset as the count of flaky tests, seen in Table III, is sparse for most configurations, with one flaky test per configuration being common. One of the programs where it shows noticeable results are for *CorfuDB*. *CorfuDB*, as listed in Table IV has relatively more flaky tests when compared to the other programs. This varies from 10 to 75 flaky tests. As depicted in Figure 1 for *CorfuDB*, flakiness across runs varies for the different instrumentations.

B. Discussion of Findings

Our experiment shows that while instrumentation may cause flakiness, this effect is rare. There are only a few cases, which show an impact of instrumentation on the presence of flaky tests. There is no unified pattern that we could identify with regard to specific instrumentation tools or configurations that would introduce/increase flakiness across the programs we study.

A related question is whether instrumentation can interfere with existing flakiness detection and prevention techniques. Given a large number of such techniques [1], [2], this discussion is incomplete.We aim to investigate this question further in our future research.

For *detection techniques based on static analysis* such as [13], interference is possible (but may still be unlikely) simply because the instrumentation is not part of the analyses, and this may lead to additional false positives and/or false negatives. This effect can be difficult to mitigate.

Dynamic prevention and detection techniques such as *RootFinder* [7] and *saflate* [15] (both use instrumentation) can be prone to instrumentation order dependencies. While it is possible to craft examples showing this, it is unlikely to occur in practice.

On the other hand, dynamic prevention and detection techniques that use a particular platform (such as *NonDex* [17] using a modified standard library or VMVM [18]) may also be sensible to instrumentation, as the instrumentation code itself is affected by those changes.

VI. THREATS TO VALIDITY

Even though the results of our experiments indicate the effects of instrumentation on test flakiness, there are threats to generalising them. First, the set of programs may not be representative, which we address using a set of programs from a previous flakiness study. As flaky tests can be non-deterministic and caused by environmental factors, results may vary if the experiment is repeated. To account for such variance, we have executed the experiment 20 times and fixed the factors we control, such as the hardware, OS and JVM. Nonetheless, the results could change with more reruns of the experiment.

VII. CONCLUSION AND FUTURE WORK

In this paper, we discuss the possible impact of instrumentation on the presence of flakiness in test suites. We hypothesised that instrumentation does have an impact on the presence or frequency of flaky tests. To investigate this, we conducted an experiment using five Java instrumentation tools representing three different instrumentation types (APM, coverage and profiling). Our results from studying nine open-source programs show that instrumentation has little to no effect on the presence or frequency of flaky tests (i.e., it does not increase flakiness). These are preliminary results, and a more comprehensive study is needed. In the future, we plan to extend this study to include a more significant number of instrumentation tools and to study programs with a larger number of reruns.

ACKNOWLEDGMENT

This work is funded by Science for Technological Innovation (SfTI) National Science Challenge (NSC) of New Zealand, grant number MAUX2004.

REFERENCES

[1] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.

[2] S. Rasheed, A. Tahir, J. Dietrich, N. Hashemi, and L. Zhang, "Test flakiness' causes, detection, impact and responses: A multivocal review," *arXiv preprint arXiv:2212.00908*, 2022.

[3] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in javascript," in *Proceedings of the 38th IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022.

[4] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 643–653.

[5] K. Costa, R. Ferreira, G. Pinto, M. d'Amorim, and B. Miranda, "Test flakiness across programming languages," *IEEE Transactions on Software Engineering*, 2022.

[6] M. Ivanković, G. Petrović, R. Just, and G. Fraser, "Code coverage at google," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019.

[7] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019.

[8] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "Deflaker: Automatically detecting flaky tests," in *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 2018, pp. 433–444.

[9] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 211–224.

[10] D. Silva, L. Teixeira, and M. d'Amorim, "Shake it! detecting flaky tests caused by concurrency with shaker," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 301–311.

[11] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "ifixflakies: A framework for automatically fixing order-dependent flaky tests," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 545–555.

[12] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1572–1584.

[13] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Transactions on Software Engineering*, 2022.

[14] D. Tengeri, F. Horváth, A. Beszédes, T. Gergely, and T. Gyimóthy, "Negative effects of bytecode instrumentation on java source code coverage," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 1, 2016, pp. 225–235.

[15] J. Dietrich, S. Rasheed, and A. Tahir, "Flaky test sanitisation via on-the-fly assumption inference for tests with network dependencies," in *2022 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2022.

[16] M. Cordeiro, D. Silva, L. Teixeira, B. Miranda, and M. d'Amorim, "Shaker: a tool for detecting more flaky tests faster," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021.

[17] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "Nondex: A tool for detecting and debugging wrong assumptions on java api specifications," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016.

[18] J. Bell and G. Kaiser, "Unit test virtualization with vmvm," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, pp. 550–561.